Raising Exceptions

You can also force Python to `raise` exceptions using the raise statement. In the raise statement, you specify the exception to be raised and can also clarify why that exception was raised.

```
try:
    a = int(input("Enter a positive integer to multiply by itself: "))
    if a>=0:
        print (a*a)
    if a <= 0:
        raise ValueError("That is not a positive number!")
except ValueError as ve:
    print(ve)
```

If the user enters any negative number, the exception `ValueError` will be raised and the message `That is not a positive number` will be displayed and if enters any positive integer, it will multiply by itself.

User Defined Exceptions

In spite of the wide variety of inbuilt exceptions in Python, sometimes it's necessary to create your own exceptions. These type of exceptions are known as user defined exceptions.

To create an exception, you need to create a new class. This exception class has to be derived, either directly or indirectly, from `Exception` class. Look at this code

```
class Error(Exception):
   """Base class for other exceptions"""
   pass

class ValueTooSmallError(Error):
   """Raised when the input value is too small"""
   pass

class ValueTooLargeError(Error):
   """Raised when the input value is too large"""
   pass
```